CROWDVIEW · BACKEND HANDBOOK

CrowdView 백엔드 교본

이 프로젝트에서 실제로 쓰고 있는 서버·저장소·외부 연동·운영 도구를 개념 → 역할 → 우리 코드에서의 모습 → 직접 운영으로 옮길 때 순서로 정리한 학습용 문서입니다.

2026-10-22 기준 · main #199 · 운영 https://crowdview.app
읽는 법: 각 장 첫 상자(개념) → 표(역할) → "우리 코드" → "SA 관점" 순서로 읽으면 됩니다.

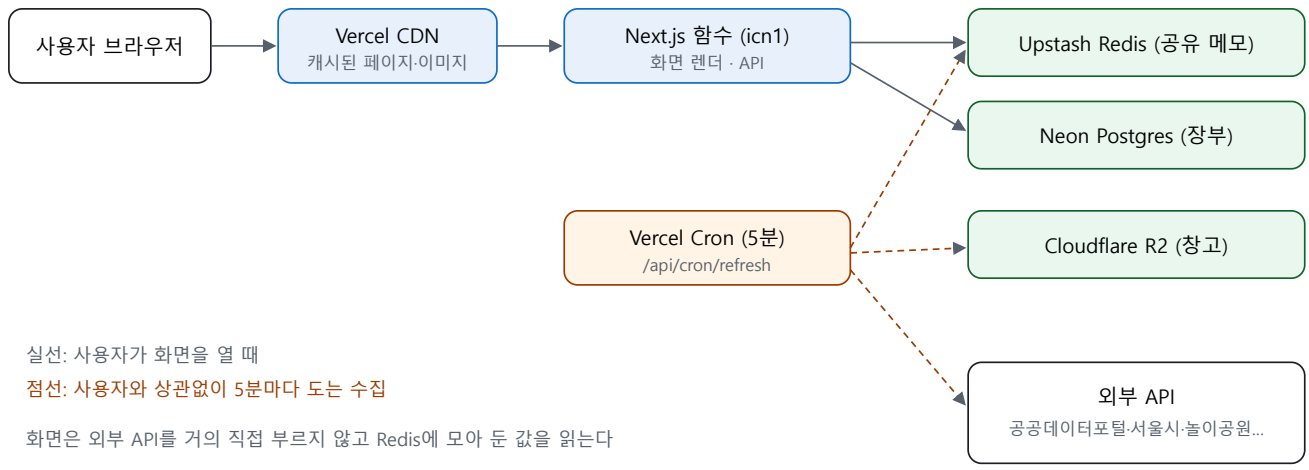
차례

1. 큰 그림: 요청 하나가 지나가는 길
2. 실행 환경: Vercel · Next.js · 함수 · CDN
3. 저장소 5종: 메모리 · Redis · Postgres · R2 · 저장소 안 파일
4. 데이터 수집 파이프라인: Cron → 수집 → 저장 → 보관 → 채점
5. 외부 API를 다루는 기술: 캐시 · 한도 · 실패 · 빌드
6. 로그인 · 결제 · 알림 · 메일
7. 다국어와 번역
8. 관측과 품질: health · CI · 테스트 · 보고서
9. 보안과 비밀값
10. 비용 구조와 한도 한눈에
11. SA 관점: 직접 운영 클라우드로 옮긴다면
12. 용어집

> **한 문장 요약**
> CrowdView는 "Vercel 위의 Next.js 앱"이 5분마다 공공 API에서 혼잡 자료를 모아 **Redis(빠른 공유 메모)**에 두고, 회원·결제는 **Postgres(장부)**에, 원본 기록은 **R2(창고)**에 쌓는 구조입니다.

1. 큰 그림: 요청 하나가 지나가는 길

> **개념**
> 웹 서비스의 백엔드는 "사용자 요청에 답하는 길"과 "사용자와 상관없이 뒤에서 도는 일" 두 갈래입니다. CrowdView는 두 번째(수집)가 핵심이고, 첫 번째(화면)는 수집해 둔 것을 빠르게 보여 주는 역할입니다.



사용자 쪽 길 (읽기)

1. 브라우저가 `crowdview.app/place/lotteworld`를 요청합니다.
2. Vercel CDN에 5분 안에 만든 페이지가 있으면 그대로 돌려줍니다(ISR). 이 경우 서버 코드는 아예 실행되지 않습니다.
3. 없거나 오래됐으면 Next.js 함수가 깨어나 Redis에서 최신 혼잡도를 읽고 페이지를 다시 만듭니다.
4. 로그인·알림·결제 같은 개인 정보가 필요하면 Postgres를 읽습니다.

뒤쪽 길 (쓰기)

1. Vercel Cron이 5분마다 `/api/cron/refresh`를 부릅니다.
2. 장소마다 "지금 다시 받을 차례인지"(적응형 주기)를 보고 외부 API를 부릅니다.
3. 계산한 혼잡도를 Redis에 저장하고, 같은 내용을 R2에 압축 파일로 보관합니다.
4. 정시에는 예측 정확도를 채점하고 모델 보정값을 갱신합니다.

> **왜 이렇게 나눴나**
> 공공 API는 느리고 하루 호출 한도가 있습니다. 사용자 수만큼 API를 부르면 한도가 금방 찹니다. 수집을 한곳(Cron)에 모으면 사용자가 1명이든 10만 명이든 외부 호출 수는 같습니다.

2. 실행 환경: Vercel · Next.js · 함수 · CDN

> **개념**
> **PaaS(Platform as a Service)**는 서버를 직접 빌리지 않고 코드만 올리면 실행·확장·인증서·배포를 대신해 주는 플랫폼입니다. Vercel이 그 예이고, 그 위에서 도는 웹 프레임워크가 Next.js입니다.

구성요소	역할	우리 프로젝트에서
Next.js 16 (App Router)	화면(React)과 서버 API를 한 저장소에서 만든다. 폴더 구조가 곧 주소	`src/app/[lang]/…` 화면, `src/app/api/…` API, `src/proxy.ts` 언어 라우팅
Vercel Functions (Fluid Compute)	요청이 올 때 깨어나는 서버 코드. 서버 대수는 자동 조절. Node.js 런타임	지역 `icn1`(서울, AWS 기반). 수집 Cron, API, 페이지 다시 만들기
CDN (Edge Network)	전 세계 가까운 지점에 응답을 복사해 두고 바로 준다	페이지·이미지·폰트. `Cache-Control` 헤더로 수명 지정
ISR (Incremental Static Regeneration)	미리 만든 페이지를 정해진 주기마다 뒤에서 새로 만든다	홈·장소 화면 `revalidate = 300`(5분). 빌드 때 한국어 장소 화면만 미리 만든다
Vercel Cron	정해진 시각에 주소를 호출해 주는 예약 실행	`vercel.json`: 수집 `*/5 * * * *`, 비용 보고서 `0 0 * * *`(09시 KST)
환경변수	비밀값·설정을 코드 밖에 둔다. Production/Preview/Development로 나뉨	API 키, DB 주소, R2 키. 값이 없으면 기능이 꺼지는 구조(R2·하이원·SK)
배포	main에 머지하면 Production 빌드·배포. Ignored Build Step으로 문서만 바뀐 커밋은 건너뜀	`scripts/vercel-ignore.sh`. Preview는 PR마다(대부분 건너뜀)

빌드 시간과 실행 시간은 다르다

빌드(next build)는 배포 직전에 한 번, 여러 작업자 프로세스가 페이지를 미리 만듭니다. **실행**은 배포 뒤 요청이 올 때입니다. 빌드 중에 외부 API를 부르면 배포할 때마다 호출이 몰립니다. 그래서 우리 코드는 빌드 중에 Redis에 쓰지 않고(`lib/cache.ts`의 buildSafe), Papago를 부르지 않고, 저장된 값이 있으면 외부 API도 부르지 않습니다(#199).

> **10/21에 실제로 겪은 일**
> 운영 배포가 하루 51번 있었고, 빌드마다 장소 화면 117개를 만들면서 에어코리아·TourAPI를 다시 불렀습니다. 그 결과 호출이 주기보다 2~13배 많았습니다. 교훈: 서버리스에서 "빌드도 호출자"라는 점을 잊으면 안 됩니다.

Next.js에서 자주 나오는 말

- **Server Component**: 서버에서만 실행되는 화면 조각. 비밀 키를 써도 브라우저로 안 나간다. 우리 화면 대부분이 이것
- **Client Component** (`"use client"`): 브라우저에서 실행. 버튼·지도·차트. 시간대·언어를 서버와 맞춰야 한다(서버는 UTC·영어)
- **Route Handler**: `route.ts` 파일 하나가 API 하나. 예: `/api/health`, `/api/cron/refresh`
- **proxy.ts**(옛 middleware): 모든 요청 앞에서 언어 주소(/en, /ja…)를 정하고 리다이렉트

3. 저장소 5종: 무엇을 어디에 두나

> **개념**
> 저장소는 "얼마나 빨라야 하나", "얼마나 오래 둬야 하나", "얼마나 큰가", "관계·정합성이 중요한가" 네 질문으로 고릅니다. 한 저장소로 다 하려고 하면 비싸거나 느리거나 위험해집니다.

저장소	비유	성격	우리가 두는 것	수명
프로세스 메모리	책상 위 메모	가장 빠름. 서버(인스턴스)마다 따로, 꺼지면 사라짐	방금 읽은 Redis 값, 진행 중 요청 묶기	초~분
Upstash Redis	사무실 공용 화이트보드	키-값, 밀리초 응답, 모든 서버가 공유, 키마다 만료(TTL)	수집한 혼잡도, API 응답 캐시, 호출 수 집계, 잠금, 정확도·백분위 자료	분~400일
Neon Postgres	회계 장부	관계형 SQL, 트랜잭션, 정합성, 스키마(마이그레이션)	회원·세션, 이용권·결제, 알림, 찜, 일정, 제보	영구
Cloudflare R2	창고	객체 저장(S3 호환). 파일 단위, 크고 싸다, 바깥으로 내보내기 무료	5분마다 혼잡도 원본 NDJSON.gz	영구(쌓기만)
저장소 안 파일	인쇄된 안내 책자	코드와 같이 배포되는 JSON. 호출 비용 0, 바꾸려면 배포	명소 117곳, 서울 화장실·와이파이, 사진 목록 (`src/data`)	배포 단위

3-1. Upstash Redis — 공유 메모

Redis는 메모리에 키-값을 두는 데이터베이스입니다. 서버리스 함수는 요청마다 다른 서버에서 실행될 수 있어서, 서버끼리 공유해야 하는 "지금 상태"를 Redis에 둡니다. Upstash는 Redis를 HTTP(REST)로 쓰게 해 주는 서비스라, 연결을 오래 유지할 수 없는 서버리스와 잘 맞습니다.

키 모양	용도	코드
`dg:/경로:해시`	공공데이터포털 응답 캐시 (받은 시각 포함)	lib/cached, datagokr
`{키}:fail`, `{키}:failn`	원본 실패 표시와 연속 실패 수 (5분→15분→1시간)	lib/cached
`quota:날짜:API`	API별 하루 호출 수 (90%에서 멈춤)	lib/api-quota
`seoul:live:all`	서울 121곳 실시간 인구 묶음	lib/seoul-live
`acc:*`, `pct:*`, `calib:*`	예측 채점, 백분위 분포, 장소별 보정 칸	lib/accuracy, crowd/percentile, crowd/calibration
`papago:blocked`, `tr*`	번역 차단 표시, 번역 결과·잠금	lib/name-translate
`archive:last`	마지막 R2 보관 파일 (health 표시)	lib/archive

핵심 명령 4개

- `GET/SET key value EX 초`: 읽기·쓰기와 만료
- `MGET`: 여러 키를 명령 1번에 (비용 절약)
- `SET NX`: 키가 없을 때만 쓰기 → **분산 잠금**(여러 서버가 같은 일을 동시에 하지 않게, 정시 채점이 한 번만 돌게)
- `INCR`: 숫자 1 올리기 → **호출 수 집계**

> **비용 단위는 "명령 수"**
> 무료는 월 50만 명령입니다. 10/19에 빌드가 Redis를 두드려 한도를 넘긴 적이 있습니다(#127로 수정). 그래서 "시간대별 집계"처럼 명령을 크게 늘리는 기능은 신중히 넣습니다.

3-2. Neon Postgres — 장부

관계형 데이터베이스는 표(테이블)끼리 관계를 맺고, 여러 변경을 한 묶음(트랜잭션)으로 처리해 "돈은 빠졌는데 이용권은 안 생기는" 일을 막습니다. Neon은 서버리스용 Postgres로, 쓰지 않을 때 잠들어 비용이 적습니다.

- 스키마는 `db/migrations/001_auth.sql` ~ `008_plus_pass.sql` 순서로 쌓고 `npm run db:migrate`로 적용
- 로그인(Better Auth)의 user·session·account, Plus 이용권·결제(Toss), 알림·찜·일정(trip_watch)·혼잡 제보(crowd_report)
- 접속은 `DATABASE_URL` 하나, 드라이버는 `pg`

> **Redis와 Postgres를 가르는 기준**
> 잃어버려도 다시 만들 수 있으면 Redis, 잃어버리면 사고면 Postgres. 혼잡도는 5분 뒤 다시 받으면 되지만, 결제 기록은 다시 만들 수 없습니다.

3-3. Cloudflare R2 — 창고 (10/22 가동)

객체 저장소는 파일을 "버킷" 안에 "키(경로)"로 넣고 빼는 저장소입니다. 수정보다 쌓기에 맞고, 용량 단가가 매우 쌉니다. R2는 AWS S3와 같은 API를 써서, 나중에 S3나 MinIO로 옮길 때 주소(endpoint)만 바꾸면 됩니다.

```
archive/crowd/2026/10/22/1010.ndjson.gz   ← 5분마다 1개, 하루 288개 (약 3MB)
줄마다 JSON 하나: {"kind":"place","at":"…","placeId":"everland","score":55,…}
                  {"kind":"seoul","at":"…","code":"POI014","area":"강남
역","min":30000,"max":32000,…}
```

- **NDJSON**: 한 줄에 JSON 하나. 줄 단위로 이어 붙이고 나눠 읽기 쉬워 로그·분석용 표준
- **gzip**: 텍스트를 1/10 안팎으로 압축
- **왜 필요한가**: 원천 API는 "지금 값"만 주고 과거를 안 준다. 우리가 쌓지 않으면 모델 학습·정확도 검증 자료가 없다
- 읽기 도구: `npm run archive:read -- 2026-10-22` → 하루치 합치기 + 장소별·지역별 평균·최대 요약

3-4. 저장소 안 파일과 public 폴더

자주 안 바뀌는 자료(화장실 2,015곳, 명소 소개, 사진 목록)는 주 1회 스크립트(`scripts/seoul-data/sync.mjs`, GitHub Actions `seoul-data.yml`)로 받아 JSON으로 저장소에 넣습니다. 사이트는 이 파일만 읽어 API 호출과 서버 비용이 0입니다. 이미지·폰트는 `public/`에 두고 CDN이 나눠 줍니다.

4. 데이터 수집 파이프라인

> **개념**
> **파이프라인**은 "받기 → 다듬기 → 저장 → 활용"을 정해진 순서로 반복하는 흐름입니다. 우리 파이프라인은 5분마다 한 바퀴 돕니다.

Vercel Cron 5분 → **적응형 주기 판단** → **원천 호출** → **혼잡도 계산** → **Redis 저장** → **R2 보관** → **정시: 채점·보정**

단계	하는 일	코드
적응형 주기	놀이공원 운영 중 5분, 서울·공항 10분, 문 닫은 시간·추정 장소 30분. 붐빌 때 촘촘히, 필요 없을 때 드물게	lib/crowd/cadence
원천 호출	서울시 도시데이터, 인천·한국공항공사, ThemeParks.wiki·Queue-Times(놀이기구 대기), 기상청·에어코리아 등	lib/crowd/provider, signals, themeparks
혼잡도 계산	실측이 있으면 실측, 없으면 모델(요일×시간 패턴 × 날씨·공휴일·축제·검색 추세 등). 5단계로 표시	lib/crowd/model, service, levels
저장	장소별 스냅샷과 지난 시간 이력을 Redis에	lib/crowd/history
보관	같은 스냅샷 + 서울 121곳을 R2에 압축 보관	lib/archive
채점	1·3·24시간 전 예측과 실제를 비교해 정확도 기록 (`/accuracy`)	lib/accuracy
보정	장소별 평일/주말×시간 칸마다 모델이 틀린 비율을 배워 다음 예측에 곱함	lib/crowd/calibration
백분위	최근 4주 같은 요일·시간 분포에서 지금 값의 위치(평소보다 붐빔/한산)	lib/crowd/percentile

> **멱등성(idempotency)**
> Cron이 실수로 두 번 돌아도 결과가 같아야 합니다. 정시 작업은 `SET NX` 잠금으로 "이 시각 채점은 한 번만"을 보장합니다. 분산 시스템의 기본기입니다.

매일 비용 보고서

09시 KST Cron이 전날 API 호출 수·NCP 요금 등을 모아 PDF로 만들고(`lib/billing/pdf`) Discord 웹훅과 Resend 메일로 보냅니다. "API 한도 70% 이상" 경고도 여기서 나옵니다.

5. 외부 API를 다루는 기술

> **개념**
> 외부 API는 느리고, 가끔 죽고, 한도가 있고, 늦게 갱신됩니다. 좋은 백엔드는 이 넷을 전제로 설계합니다. 우리 코드는 이걸 `lib/cached.ts`와 `lib/api-quota.ts` 두 곳에 모았습니다.

5-1. 3단 캐시 (cached)

```
메모리(이 서버) ─없거나 오래됨─▶ Redis(모든 서버) ─없거나 오래됨─▶ 원본 API
   fresh 안이면 바로 반환          fresh 안이면 반환·메모리에 복사      받아서 둘 다에 저장
```

- **fresh**: 이 시간 안에는 원본을 다시 부르지 않음 (에어코리아 1시간, 기상청 중기 6시간, 요금표 24시간)
- **stale 보관**: fresh가 지나도 최소 6시간은 값을 남겨, 원본이 죽으면 마지막 값을 쓴다 (stale-while-error)
- **빈 결과**: 발표 전이라 비어 있으면 30분만 기억하고 이전 값을 쓴다
- **in-flight 묶기**: 같은 요청이 동시에 들어오면 원본 호출은 하나만

5-2. 실패 다루기

- **타임아웃**: 공공데이터포털 8초, 국가교통정보 15초. 무한정 기다리지 않는다
- **실패 공유**: 실패하면 Redis에 표시해 다른 서버도 잠시 부르지 않음 (새 서버마다 8초씩 기다리는 일 방지)
- **지수적 물러서기(backoff)**: 연속 실패면 5분 → 15분 → 1시간 (#199)
- **동시 호출 제한**: 공공데이터포털은 한 키로 동시에 많이 부르면 느려져서 최대 6개 (세마포어)

5-3. 한도(쿼터) 지키기

- 원본을 실제로 부를 때만 `INCR quota:날짜:API` (캐시 적중은 안 셈)
- 한도의 90%에 닿으면 그날은 더 안 부르고 마지막 값 사용 (QuotaError)
- 원본이 "한도 초과"라고 답하면 그날 남은 호출을 막음
- Papago처럼 월 상한이 있는 것은 월 집계도 같이, 429를 받으면 6시간 전 서버 차단
- 한도 값은 `APIS` 표 한곳에서 관리 (10/22 운영계정 값으로 갱신)

5-4. 늦게 오는 자료

원천	늦는 정도	처리
서울 실시간 도시데이터	20~40분	관측 시각을 같이 표시, 90분 넘으면 버림
기상청 단기예보	발표 후 약 10분	직전 발표 3개까지 거슬러 시도
에어코리아 실시간	정각 측정 후 15~30분	측정 시각(dataTime) 함께 저장, "-"는 값 없음 처리

놀이공원 대기	약 5분	ThemeParks.wiki 실패 시 Queue-Times로 대체

5-5. 고정 자료 모드 (테스트용)

`DATA_FIXTURES=normal|stale|invalid|empty`로 켜면 외부 API 대신 `tests/fixtures`의 파일을 씁니다. CI의 e2e 시험이 외부 상황에 흔들리지 않게 하는 장치입니다(테스트 더블).

6. 로그인 · 결제 · 알림 · 메일

로그인: Better Auth + OAuth

OAuth는 "카카오가 이 사람이 맞다고 보증"해 주는 방식이라 우리는 비밀번호를 보관하지 않습니다.

- 카카오·네이버·구글, 콜백 `/api/auth/callback/…`
- 세션은 Postgres, 쿠키로 식별
- `BETTER_AUTH_SECRET`로 서명

결제: 토스페이먼츠

- 지금은 1개월 이용권(일반 결제). 자동 갱신(빌링)은 퀵오픈 미지원이라 숨김 (`NEXT_PUBLIC_PLUS_BILLING=off`)
- 흐름: 결제창 → 성공 주소로 돌아옴 → 서버가 `/v1/payments/confirm`으로 **승인 확정** → Postgres에 이용권 기록
- 금액 위변조 방지: 서버가 주문 금액을 다시 확인

알림: Web Push

- 브라우저가 준 구독 정보를 저장했다가, 찜한 곳이 한산해지면 `web-push`로 발송
- VAPID 키로 "우리가 보낸 알림"임을 서명
- 무료 1개, Plus 30개

메일·운영 알림

- Resend(무료 하루 100통): 비용 보고서
- Discord 웹훅: 비용 보고서 PDF, 장애 알림

7. 다국어와 번역

- **주소 기반 언어**: 한국어는 주소 그대로, 영어·일본어·중국어(간체·번체)는 `/en` `/ja` `/zh-CN` `/zh-TW`. `proxy.ts`가 쿠키(`cv-lang`)와 접속 국가로 첫 방문 언어를 정한다(검색 봇은 제외)
- **화면 문구**: `scripts/i18n/ui.mjs`에 원문과 번역을 두고 `build.mjs`로 사전 파일 생성. 실행 중 번역 호출 없음
- **고유명사(장소·주소)**: Papago로 한 번 번역해 Redis에 오래 저장. 운영에서만, 빌드 중 금지, 하루·월 상한, 429면 6시간 차단

> **10/21 Papago 사고**
> Preview 배포가 빌드마다 전부 재번역했는데, 그 호출은 메모리 캐시라 집계에 안 잡혀 NCP 월 한도를 넘겼습니다. 교훈: "세는 곳"과 "쓰는 곳"이 같아야 하고, 모든 환경이 같은 집계를 봐야 합니다.

8. 관측과 품질

> **개념**
> **관측 가능성(observability)**은 "지금 시스템이 건강한지 밖에서 알 수 있는 정도"입니다. 모니터링 화면, 자동 시험, 알림이 여기에 속합니다.

도구	무엇을 보나	언제
`/api/health`	Redis 연결, 원천별 신선도(oldestMin), 한도 70% 넘은 API(quotaHot), Papago, R2 보관, DB 마이그레이션, 키 점검(keyProbe)	항상. 503이면 이상
CI `ci.yml`	타입 검사 + 단위 테스트 100개(node:test) + Playwright e2e(고정 자료, axe 접근성)	모든 PR
smoke `smoke.yml`	운영 사이트 주요 화면·API가 열리는지 + Lighthouse 점수	배포 뒤·매일
야간 운영 e2e	운영 주소로 e2e 전체	매일 03시 KST
Web Vitals	실제 사용자의 로딩 속도 지표 수집(`/api/vitals`)	상시
비용 보고서	API 호출·요금·한도 경고	매일 09시
Sentry	오류 추적 (DSN 미설정, 꺼짐)	—

테스트의 층

- **단위 테스트**: 순수 계산 함수(분위수, 보정, 캐시 규칙)를 빠르게. `tests/*.test.ts`
- **E2E 테스트**: 실제 브라우저로 화면을 눌러 본다. `e2e/*.spec.ts`, 모바일·사파리·데스크톱 3종
- **접근성 테스트(axe)**: 글자 대비 4.5:1 등. 오늘 "늘어나는 중" 빨간 글씨가 시간대에 따라 걸린 것도 이것

9. 보안과 비밀값

- **비밀값은 환경변수에만**: 코드·로그·채팅에 내지 않는다. Vercel에서는 Sensitive로 저장(밖에서 다시 못 읽음)
- **브라우저로 나가는 값**: `NEXT_PUBLIC_`으로 시작하는 변수만 브라우저에 포함(지도 Client ID 등). 이름 실수로 비밀이 새지 않게 주의
- **Cron 보호**: `/api/cron/*`는 `CRON_SECRET` 헤더가 있어야 실행
- **속도 제한(rate limit)**: 알림·찜·제보·관리자 API 등에 요청 횟수 제한 (`lib/ratelimit`)
- **최소 권한**: R2 토큰은 버킷 하나에 읽기·쓰기만. 공공 API 키는 서버에서만 사용
- **방화벽·허용 IP**: 국가교통정보센터처럼 해외 IP를 막는 곳은 고정 출발지 IP가 필요 → 집 데스크탑(국내 회선)이 받아 서버로 밀어 넣는 구조로 해결 예정
- **개인정보**: 위치는 서버에 저장하지 않음, 사업자등록증·사진 폴더는 커밋 금지

10. 비용 구조와 한도 한눈에

서비스	과금 단위	현재 요금제·한도	주의
Vercel	월 정액 + 사용량	Pro, Spend Management $50	함수 실행 시간·빌드 횟수
Upstash Redis	명령 수	무료 월 50만 (10/19 초과 이력)	빌드·집계 키 추가 시
Neon Postgres	저장 용량·연산 시간	무료 0.5GB	로그성 자료는 넣지 않기
Cloudflare R2	저장 GB·쓰기 횟수	무료 10GB·월 100만 쓰기	1년 약 1GB, 월 쓰기 약 8,600
공공데이터포털	API별 하루 호출	40건 모두 운영계정(33 승인): 1만 ~100만/일	관광공사 4개 승인 대기 (1,000)
Papago (NCP)	글자 수	우리 상한 하루 3만·월 30만 자	11/1까지 차단
SK 혼잡도	건당 33원	하루 30건 상한	환경변수로 조절
Resend	메일 수	무료 하루 100·월 3,000	—

자세한 API별 표는 `docs/API-한도-2026-10-22.md`.

11. SA 관점: 직접 운영 클라우드로 옮긴다면

> **설계 원칙 (사용자 목표: 지금은 Vercel, 나중엔 직접 운영)**
> 처음부터 "특정 회사 기능에 묶이지 않는" 방식을 골랐습니다. Next.js 데이터 캐시 대신 Redis 캐시(lib/cached), Vercel Blob 대신 S3 호환 R2, 표준 SQL Postgres를 쓴 이유입니다.

지금	AWS로 옮기면	직접 운영(온프레미스·VM)	바꿀 곳
Vercel Functions + CDN	ECS Fargate/Lambda + CloudFront	Docker 컨테이너(Node) + Nginx	`next start`, 빌드 설정
Vercel Cron	EventBridge Scheduler	cron / Kubernetes CronJob	호출 주소만 같으면 됨
Upstash Redis	ElastiCache (Redis/Valkey)	Redis 컨테이너	`lib/cache.ts` 접속부
Neon Postgres	RDS / Aurora Postgres	Postgres 컨테이너	`DATABASE_URL`
Cloudflare R2	S3	MinIO	`R2_ENDPOINT`
환경변수	Secrets Manager / SSM	.env + Vault	주입 방식
GitHub Actions	그대로 또는 CodePipeline	그대로 또는 Jenkins	배포 단계

공부 포인트

1. **상태 없는 서버(stateless)**: 서버는 언제든 죽고 늘어난다. 상태는 Redis·Postgres·R2에. 그래서 수평 확장이 쉽다
2. **캐시 계층**: CDN → 메모리 → Redis → 원본. 각 층의 수명(TTL)을 정하는 것이 설계
3. **읽기/쓰기 분리**: 쓰기(수집)는 한곳에서 주기적으로, 읽기(화면)는 많이. CQRS의 아주 단순한 형태
4. **장애 격리**: 원천 하나가 죽어도 마지막 값·대체 원천·모델 값으로 화면은 산다 (graceful degradation)
5. **비용 관측**: 클라우드 비용은 "호출 수×단가". 집계를 먼저 만들고 기능을 붙인다
6. **네트워크 경계**: 허용 IP·고정 출구 IP(NAT, Static IP)·해외 IP 차단 같은 문제는 SA가 자주 만나는 주제

12. 용어집

용어	뜻
PaaS / 서버리스	서버 관리 없이 코드만 올리면 실행·확장해 주는 방식. 요청이 없으면 비용도 거의 없음
CDN	응답을 사용자 가까운 지점에 복사해 두는 전 세계 캐시망
ISR	정적 페이지를 주기적으로 뒤에서 다시 만드는 Next.js 기능
TTL	값이 자동으로 사라지기까지 남은 시간
캐시 적중/실패 (hit/miss)	캐시에 있어 바로 준 경우 / 없어서 원본을 부른 경우
stale	유효기간은 지났지만 아직 쓸 수 있는 묵은 값
분산 잠금	여러 서버 중 하나만 어떤 일을 하도록 Redis 등으로 표시하는 것
멱등성	같은 일을 여러 번 해도 결과가 한 번 한 것과 같은 성질
백오프	실패할수록 재시도 간격을 늘리는 것
쿼터	정해진 기간에 허용된 호출·사용량 한도
마이그레이션	데이터베이스 구조 변경을 순서 있는 파일로 기록·적용하는 것
트랜잭션	여러 변경을 전부 하거나 전부 안 하는 묶음
객체 저장소 / 버킷	파일을 키로 넣고 빼는 저장소 / 그 저장 공간 단위
NDJSON	한 줄에 JSON 객체 하나씩 쓴 텍스트 형식
OAuth	다른 서비스(카카오 등)가 사용자 신원을 대신 보증해 주는 로그인 표준
웹훅	일이 생기면 정해진 주소로 자동으로 알려 주는 HTTP 호출
Cron 식	`*/5 * * * *` = 5분마다. 분·시·일·월·요일 순 (UTC 기준)
E2E / 단위 테스트	사용자처럼 처음부터 끝까지 시험 / 함수 하나만 시험
Fixture	테스트에서 진짜 대신 쓰는 고정 자료
Static IP / NAT	나가는 주소를 고정하는 것 / 내부 주소를 공인 주소로 바꿔 내보내는 장치